

chicago_tnp_demand_forecast

June 29, 2026

1 Chicago TNP Trips — Daily Demand Forecast

Predict **daily trip demand per area (H3 cell)** for the busiest cells, using calendar and past-demand features. A linear baseline is compared against LightGBM.

Data source: MatrixOne (`chicago_tnp.trips`), Chicago rideshare trips, 100k/month sample.

```
[1]: # Install dependencies (run once)
%pip install -q pandas numpy scikit-learn lightgbm pymysql
```

[notice] A new release of pip is

available: 24.0 -> 26.1.2

[notice] To update, run:

```
pip3 install --upgrade pip
```

Note: you may need to restart the kernel to use updated packages.

1.1 1. Build the target: daily demand per cell

Count trips per (cell, day). We use **daily** granularity (not hourly): the data is a ~1.5% sample, so hourly cells are mostly noise/zeros, while daily counts are stable.

```
[2]: import pymysql
import pandas as pd

conn = pymysql.connect(host="127.0.0.1", port=6001,
                       user="root", password="111", database="chicago_tnp")

sql = """
SELECT pickup_h3, DATE(trip_start_timestamp) AS d, COUNT(*) AS demand
FROM trips
WHERE shared_trip_authorized = 0
GROUP BY pickup_h3, DATE(trip_start_timestamp)
"""

daily = pd.read_sql(sql, conn)
conn.close()

daily["d"] = pd.to_datetime(daily["d"])
print("rows:", len(daily))
```

```
daily.head()
```

```
/var/folders/jt/cz2nbl8d6vs1vcffdydk4n5440000gn/T/ipykernel_99280/3293053875.py:1
3: UserWarning: pandas only supports SQLAlchemy connectable (engine/connection)
or database string URI or sqlite3 DBAPI2 connection. Other DBAPI2 objects are
not tested. Please consider using SQLAlchemy.
```

```
daily = pd.read_sql(sql, conn)
```

```
rows: 381234
```

```
[2]:      pickup_h3      d  demand
0  89266452183ffff 2024-08-02      22
1  892664ca54fffff 2024-08-06       1
2  892664cc847ffff 2024-08-22       3
3  892664cce1bffff 2024-08-08       4
4  892664c1acfffff 2024-08-04      36
```

1.2 2. Keep the busiest cells and fill missing days with 0

Demand is very concentrated: the top 50 cells hold ~54% of all trips and have enough data to learn from. We keep those, then fill any missing (cell, day) with demand = 0.

A check (not shown here) confirmed these zeros are **real low-demand days**, scattered across all months, not contiguous data gaps — so they are kept as true zeros.

```
[3]: top_cells = (daily.groupby("pickup_h3")["demand"].sum()
                .sort_values(ascending=False).head(50).index.tolist())
daily = daily[daily["pickup_h3"].isin(top_cells)].copy()

# Full (cell x day) grid, then fill gaps with 0
days = pd.date_range(daily["d"].min(), daily["d"].max(), freq="D")
skeleton = pd.MultiIndex.from_product([top_cells, days],
                                     names=["pickup_h3", "d"]).
    to_frame(index=False)
full = skeleton.merge(daily, on=["pickup_h3", "d"], how="left")
full["demand"] = full["demand"].fillna(0).astype(int)

print("rows:", len(full), "| zero ratio:", round((full["demand"] == 0).mean() *
    100, 1), "%")
```

```
rows: 54800 | zero ratio: 12.0 %
```

1.3 3. Features

Give the model clues to predict demand: calendar, cell identity, and **past demand** (lags + rolling averages). Past demand is the strongest signal for future demand.

```
[4]: import numpy as np

full = full.sort_values(["pickup_h3", "d"]).reset_index(drop=True)
```

```

dt = full["d"].dt

# Calendar
full["dow"] = dt.dayofweek # 0=Mon ... 6=Sun
full["is_weekend"] = (dt.dayofweek >= 5).astype("int8")
full["month"] = dt.month
full["day_of_year"] = dt.dayofyear

# Cell identity (each area has a very different demand level)
full["cell_code"] = full["pickup_h3"].astype("category").cat.codes

# Lags: demand N days ago
g = full.groupby("pickup_h3")["demand"]
full["lag_1"] = g.shift(1)
full["lag_7"] = g.shift(7)
full["lag_14"] = g.shift(14)

# Rolling means. shift(1) FIRST so the window never includes today (avoids
↳ leakage).
full["roll_7"] = g.shift(1).rolling(7).mean().reset_index(level=0, drop=True)
full["roll_28"] = g.shift(1).rolling(28).mean().reset_index(level=0, drop=True)

# Each cell's baseline level, computed on the TRAIN period only (avoids leakage)
cell_mean = (full[full["d"] < "2024-01-01"].groupby("pickup_h3")["demand"]
              .mean().rename("cell_mean"))
full = full.merge(cell_mean, on="pickup_h3", how="left")

feat_cols = ["dow", "is_weekend", "month", "day_of_year", "cell_code",
             "lag_1", "lag_7", "lag_14", "roll_7", "roll_28", "cell_mean"]
full = full.dropna(subset=feat_cols).reset_index(drop=True) # drop early rows
↳ with no history
print("rows after features:", len(full))
full.head()

```

rows after features: 53400

```

[4]:
   pickup_h3      d  demand  dow  is_weekend  month  day_of_year  \
0  89266452183ffff 2022-01-29      12     5           1      1          29
1  89266452183ffff 2022-01-30       0     6           1      1          30
2  89266452183ffff 2022-01-31       9     0           0      1          31
3  89266452183ffff 2022-02-01       0     1           0      2          32
4  89266452183ffff 2022-02-02      10     2           0      2          33

   cell_code  lag_1  lag_7  lag_14  roll_7  roll_28  cell_mean
0          0    0.0    7.0    16.0  19.714286  14.678571  24.742466
1          0   12.0   45.0     1.0  20.428571  14.928571  24.742466
2          0    0.0   31.0    16.0  14.000000  14.928571  24.742466

```

3	0	9.0	44.0	0.0	10.857143	15.250000	24.742466
4	0	0.0	0.0	9.0	4.571429	14.035714	24.742466

1.4 4. Train / test split — by time, not random

We predict the future, so we train on the past (2022-2023) and test on the unseen future (2024). A **random** split would leak future info through the lag features (tomorrow's row would carry today's demand as a feature into training).

```
[5]: train = full[full["d"] < "2024-01-01"]
test = full[full["d"] >= "2024-01-01"]

X_train, y_train = train[feat_cols], train["demand"]
X_test, y_test = test[feat_cols], test["demand"]

print("train:", len(X_train), train["d"].min().date(), "->", train["d"].max().
      ↪date())
print("test :", len(X_test), test["d"].min().date(), "->", test["d"].max().
      ↪date())
```

```
train: 35100 2022-01-29 -> 2023-12-31
test : 18300 2024-01-01 -> 2024-12-31
```

1.5 5. Baseline: Linear Regression

Always build a simple baseline first, so a complex model's score has something to beat. MAE = mean absolute error = average miss in trips. We also show it as a % of mean demand.

```
[6]: from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error

lr = LinearRegression().fit(X_train, y_train)
pred_lr = np.clip(lr.predict(X_test), 0, None) # demand can't be negative
mae_lr = mean_absolute_error(y_test, pred_lr)
print(f"Linear MAE: {mae_lr:.2f} ({mae_lr / y_test.mean() * 100:.1f}% of mean_
      ↪demand)")
```

```
Linear MAE: 7.84 (23.5% of mean demand)
```

1.6 6. LightGBM

```
[7]: import lightgbm as lgb

gb = lgb.LGBMRegressor(n_estimators=300, learning_rate=0.05, random_state=42,
      ↪verbose=-1)
gb.fit(X_train, y_train)
pred_gb = np.clip(gb.predict(X_test), 0, None)
mae_gb = mean_absolute_error(y_test, pred_gb)
```

```
print(f"LightGBM MAE: {mae_gb:.2f} ({mae_gb / y_test.mean() * 100:.1f}% of_
↳mean demand)")
```

LightGBM MAE: 10.16 (30.5% of mean demand)

1.7 7. Diagnose: why doesn't LightGBM win?

Compare train vs test error for both models. This rules out classic overfitting (which would show very low train error and high test error).

```
[8]: for name, mdl in [("Linear", lr), ("LightGBM", gb)]:
      tr = mean_absolute_error(y_train, np.clip(mdl.predict(X_train), 0, None))
      te = mean_absolute_error(y_test, np.clip(mdl.predict(X_test), 0, None))
      print(f"{name:9s} train {tr:5.2f} | test {te:5.2f}")
```

```
Linear      train 22.26 | test  7.84
LightGBM    train 14.48 | test 10.16
```

```
[9]: # Which features the tree model leaned on most
imp = pd.Series(gb.feature_importances_, index=feat_cols).
      ↳sort_values(ascending=False)
print(imp)
```

```
day_of_year    2656
dow             1164
lag_14         1058
lag_7           984
lag_1           938
roll_7          608
roll_28         570
cell_mean       493
month           291
cell_code       238
is_weekend      0
dtype: int32
```

1.8 8. Summary

Setup. Predict daily trip demand per H3 cell for the 50 busiest cells. Target = trips per (cell, day), missing days filled with 0. Features: calendar, cell identity and baseline level, and past demand (lags + rolling means). Split by time: train 2022-2023, test 2024.

Result. Linear regression reaches MAE ~7.8 (about 23% of mean demand). LightGBM (MAE ~10.2) does **not** beat it.

Why LightGBM doesn't win (checked, not a bug). Features are clean (no NaN), and LightGBM is not classically overfitting (its train error is not near zero). What happens: LightGBM fits 2022-2023 more tightly, but some of that structure is specific to those years and does not hold in 2024 (the market got more stable post-pandemic — both models actually score *better* on 2024 than on the training years). The predictable signal here is mostly a stable, near-linear pattern

(“today is close to this cell’s recent average”), which the linear model captures and generalizes well. So for this task the simple model is the right choice.

Limits. Daily, top-50-cell scope (the sample can’t support hourly or rare cells). Demand = completed trips (a proxy; unmet requests are not in the data). No weather or event features yet — likely the biggest remaining lever to lower error.